ENSIA

Artificial Intelligence and Data Science

Semester 7

Introduction to Mobile Robotics

Lab Assignments

Lab 1: Introduction to ROS 2

Academic Year 2026–2027

Instructor: Dr. A. Khelloufi

Contents

Goals **3**

Prerequisites **3**

1 What is ROS? **4**

2 Why ROS? **4**

3 Basic Concepts **5**
3.1 Installing turtlesim . . . 5
3.1.1 Setting up the environment . . . 5
3.1.2 Installing turtlesim . . . 5
3.1.3 Verifying the installation . . . 5
3.1.4 Launching turtlesim . . . 6
3.2 Nodes . . . 6
3.2.1 Running nodes with `ros2 run` . . . 6
3.2.2 Listing nodes . . . 6
3.2.3 Remapping node names . . . 7
3.2.4 Inspecting nodes . . . 7
3.3 Topics . . . 7
3.3.1 Listing topics . . . 7
3.3.2 Displaying topic messages . . . 8
3.3.3 Inspecting a topic . . . 8
3.3.4 Inspecting message definitions . . . 8
3.3.5 Publishing messages from the terminal . . . 9
3.3.6 Measuring the publication rate . . . 9
3.4 Services . . . 10
3.4.1 Listing services . . . 10
3.4.2 Inspecting a service type . . . 10
3.4.3 Calling a service . . . 11
3.5 Parameters . . . 12
3.5.1 Listing parameters . . . 12
3.5.2 Reading a parameter . . . 13
3.5.3 Changing a parameter . . . 13
3.5.4 Dumping parameters . . . 13
3.5.5 Loading parameters . . . 14
3.6 Actions . . . 14

4 Creating and Building ROS 2 Packages **15**
4.1 Creating a Workspace . . . 15
4.1.1 Create a New Directory . . . 15
4.1.2 Clone a Sample Repository . . . 15
4.1.3 Resolve Dependencies . . . 16
4.1.4 Build the Workspace with Colcon . . . 16
4.1.5 Source the Overlay . . . 17
4.1.6 Modify the Overlay . . . 17
4.2 Creating a Package: Writing a Simple Publisher and Subscriber . . . 18
4.2.1 Create a Package . . . 19
4.2.2 Write the Publisher Node . . . 19
4.2.3 Add Dependencies . . . 20
4.2.4 Configure CMakeLists.txt . . . 20

4.2.5 Write the Subscriber Node . . . 21
4.2.6 Configure the Subscriber Executable . . . 21
4.2.7 Build the Package . . . 21
4.2.8 Source the Setup File . . . 22
4.2.9 Run the Publisher and Subscriber . . . 22

5 Launching Nodes **23**
5.1 Launching Multiple Turtlesim Nodes . . . 23

6 Recording and Playing Back Data **24**
6.1 Recording Data with ros2 bag . . . 24
6.1.1 Recording a Single Topic . . . 24
6.1.2 Recording Multiple Topics . . . 25
6.2 Inspecting a Recording with ros2 bag info . . . 25
6.3 Playing Back Recorded Data . . . 26

7 Using rqt, rqt_graph, and rqt_console **26**
7.1 Installing rqt . . . 26
7.2 Use rqt . . . 27
7.3 Visualizing the ROS Graph . . . 27
7.4 Monitoring Logs with rqt_console . . . 27

References **28**

Goals

By the end of this laboratory assignment, students should be able to:

- Understand the fundamental concepts of ROS 2, including nodes, topics, services, actions, and parameters, and use built-in tools to examine them.
- Set up a ROS 2 environment, create a workspace, and create a package using either Python or C++, specifying the appropriate dependencies.
- Build packages in a ROS 2 workspace using the `colcon` build tool.
- Create and execute publisher and subscriber nodes.
- Launch ROS 2 nodes and inspect their communication using command-line and graphical tools.
- Record and replay ROS 2 data using `rosbag`.

Prerequisites

Before starting this laboratory, students must have:

- ROS 2 Jazzy installed on Ubuntu 24.04.
- The `colcon` build tool installed.
- Git installed.
- A basic understanding of Linux terminal commands.
- A text editor or IDE of their choice.
- Basic C++/Python programming knowledge.

Important: Complete the ROS 2 Jazzy installation before attending the laboratory session. Refer to the provided ROS 2 installation guide.

1. What is ROS?

The Robot Operating System (ROS) is an open-source framework designed to help developers build robust and complex robotic systems.

Despite its name, ROS is not a real operating system but rather a middleware that sits between the underlaying operating system and the robot's application code.

ROS provides ready-to-use tools, libraries, and drivers for robot control, perception, navigation, and simulation. Developers can integrate sensors, actuators, and algorithms to create complete robotic systems.

ROS 2 is the next generation of ROS. It was redesigned to improve communication, security, scalability, and support for real-time robotic applications.

ROS 2 applications are organized into modular components called **nodes**. Nodes communicate using topics, services, and actions.

The **ROS graph** represents the nodes in a ROS 2 system and the communication connections between them.

For example, a mobile robot may contain:

- A camera driver node.
- A laser scanner node.
- A localization node.
- A navigation node.
- A motor controller node.

These nodes exchange information through ROS 2 communication mechanisms.

2. Why ROS?

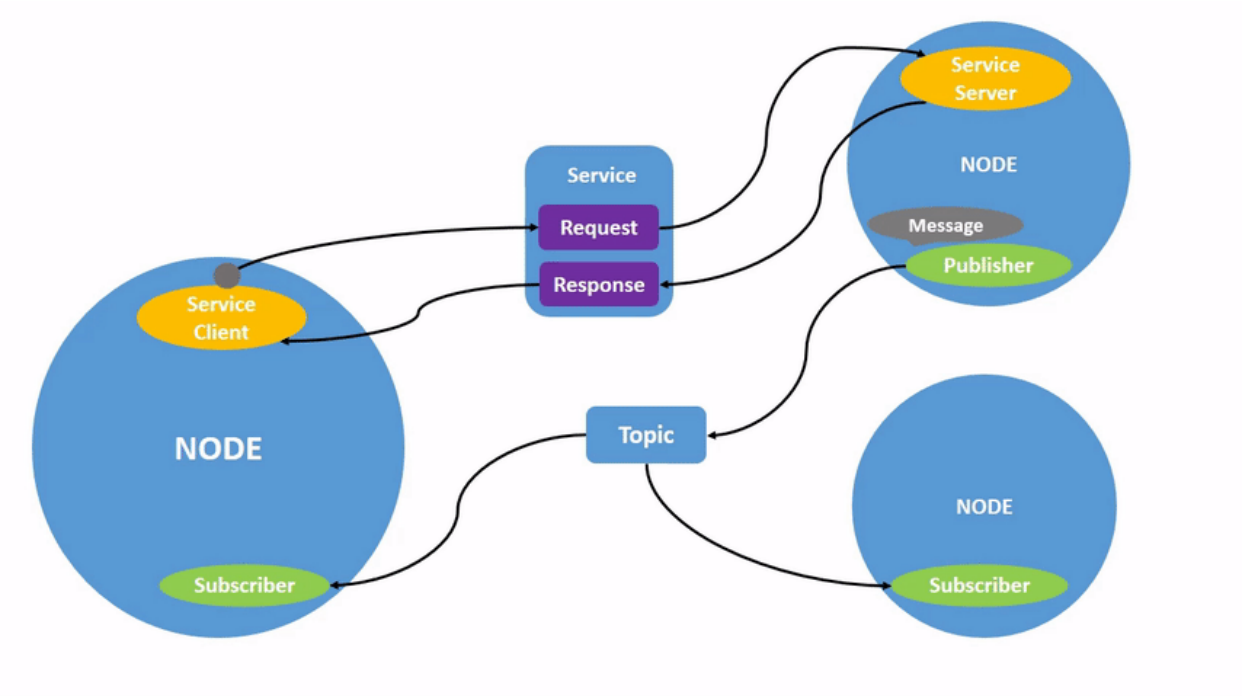
ROS has become the **de facto standard** for building, testing, and deploying robotics software. It is widely used in robotics education, research, and industrial development for several reasons.

- **Ready to use:** It provides tools, libraries, drivers, and reusable components.
- **Flexible:** It supports multiple programming languages, including Python and C++.
- **Open source:** Its software ecosystem encourages reuse, modification, and collaboration.
- **Strong community:** Supported by a large global network of developers and researchers.
- **Modular architecture:** Applications can be divided into independent nodes.
- **Proven and reliable:** Used in education, research, and real-world robots.
- **Simulation support:** ROS 2 can be integrated with robotic simulators and visualization tools.
- extbfBroad applicability: ROS is used in mobile robotics, industrial automation, aerial robotics, and other fields.

3. Basic Concepts

This section introduces the main ROS 2 concepts and demonstrates how to inspect and interact with a running ROS 2 system.

The `ros2` command-line interface allows users to start nodes, inspect topics, call services, manage parameters, and interact with other ROS 2 components.



3.1 Installing turtlesim

`turtlesim` is a lightweight simulator for learning ROS 2. It provides a simple environment for practicing with nodes, topics, services, and actions before working with a real robot or a more advanced simulator.

3.1.1 Setting up the environment

Open a terminal and source the ROS 2 Jazzy environment:

```
source /opt/ros/jazzy/setup.bash
```

This command configures the current terminal to use ROS 2 Jazzy.

3.1.2 Installing turtlesim

Execute the following commands:

```
sudo apt update
sudo apt install ros-jazzy-turtlesim
```

3.1.3 Verifying the installation

Check the available executables:

```
ros2 pkg executables turtlesim
```

The output should include:

```
turtlesim draw_square
turtlesim mimic
turtlesim turtle_teleop_key
turtlesim turtlesim_node
```

3.1.4 Launching turtlesim

To run the simulator, open a new terminal, and enter the following command:

```
ros2 run turtlesim turtlesim_node
```

A window containing a turtle should appear.

Open another terminal and launch the keyboard controller:

```
source /opt/ros/jazzy/setup.bash
ros2 run turtlesim turtle_teleop_key
```

Use the arrow keys to move the turtle.

Keep both nodes running while completing the following exercises.

3.2 Nodes

A **node** is a modular component of a ROS 2 application that performs a particular function.

For example, a node may control wheel motors, process camera images, publish laser scanner measurements, or estimate the robot's position.

A robotic system typically consists of several nodes working together.

Nodes communicate through topics, services, and actions. They can also expose parameters that configure their behavior.

3.2.1 Running nodes with `ros2 run`

The general syntax is:

```
ros2 run <package_name> <executable_name>
```

For example:

```
ros2 run turtlesim turtlesim_node
```

Here, `turtlesim` is the package name, and `turtlesim_node` is the executable name.

3.2.2 Listing nodes

The command `ros2 node list` displays the names of currently running nodes.

Open a new terminal while turtlesim is still running in the other one, and enter:

```
ros2 node list
```

The output should include:

```
/turtlesim
/teleop_turtle
```

The exact output depends on the nodes currently running.

3.2.3 Remapping node names

ROS names without modifying the source code.

Remapping allows users to reassign default node properties, like node name, topic names, service names, etc., to custom values.

Let's reassign the name of our `/turtlesim` node. In a new terminal, run the following command:

```
ros2 run turtlesim turtlesim_node \
    --ros-args --remap __node:=my_turtle
```

List the active nodes again:

```
ros2 node list
```

You should now see a node named `/my_turtle` in addition to the other running nodes.

3.2.4 Inspecting nodes

The command `ros2 node info` displays information about a node's publishers, subscribers, services, and actions.

```
ros2 node info <node_name>
```

To examine your latest node, `/my_turtle`, run the following command:

```
ros2 node info /my_turtlesim
```

Now try running the same command on the teleoperation node, and see how its connections differ from `/my_turtle`.

```
ros2 node info /teleop_turtle
```

Compare the communication interfaces of both nodes.

3.3 Topics

Topics are communication channels through which ROS 2 nodes exchange messages.

ROS 2 topics use a publish/subscribe communication model. A publisher sends messages to a topic, while subscribers receive messages from that topic.

A node can publish to several topics and subscribe to several other topics.

3.3.1 Listing topics

Running the `ros2 topic list` command in a new terminal will return a list of all the topics currently active in the system:

```
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

To display topic names and their message types:

```
ros2 topic list -t
```

For example:

```
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
```

3.3.2 Displaying topic messages

The command `ros2 topic echo <topic_name>` displays messages published on a topic

Since we know that `/teleop_turtle` publishes data to `/turtlesim` over the `/turtle1/cmd_vel` topic, let's use `echo` to introspect that topic:

```
ros2 topic echo /turtle1/cmd_vel
```

Return to the terminal where `turtle_teleop_key` is running and use the arrows to move the turtle around. Watch the terminal where your `echo` is running at the same time, and you'll see velocity data being published for every movement you make:

```
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

3.3.3 Inspecting a topic

To inspect the type and the number of publishers and subscribers, execute:

```
ros2 topic info /turtle1/cmd_vel
```

An example output is:

```
Type: geometry_msgs/msg/Twist
Publisher count: 1
Subscription count: 1
```

The counts may differ depending on the nodes currently running.

3.3.4 Inspecting message definitions

Nodes send data over topics using messages. Publishers and subscribers must send and receive the same type of message to communicate.

ROS 2 messages have predefined structures.

For example, `geometry_msgs/msg/Twist` contains linear and angular velocity vectors.

Inspect the message definition:

```
ros2 interface show geometry_msgs/msg/Twist
```

The output includes:

```
Vector3 linear
    float64 x
    float64 y
    float64 z
Vector3 angular
    float64 x
    float64 y
    float64 z
```

Each vector contains three components.

3.3.5 Publishing messages from the terminal

Now that you have the message structure, you can publish data to a topic directly from the command line using:

```
ros2 topic pub <topic_name> <message_type> '<arguments>'
```

The `'<args>'` argument is the actual data you'll pass to the topic, in the structure you just discovered in the previous section.

It's important to note that this argument needs to be input in YAML syntax. Input the full command like so:

```
ros2 topic pub --once /turtle1/cmd_vel \
geometry_msgs/msg/Twist \
"{linear: {x: 2.0, y: 0.0, z: 0.0}, \
angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

You will see your turtle move. The `-once` option publishes one message and then exits.

The turtle (and commonly the real robots which it is meant to emulate) require a steady stream of commands to operate continuously. So, to get the turtle to keep moving, you can run:

```
ros2 topic pub --rate 1 /turtle1/cmd_vel \
geometry_msgs/msg/Twist \
"{linear: {x: 2.0, y: 0.0, z: 0.0}, \
angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

The difference here is the removal of the `-once` option and the addition of the `-rate` 1 option, which tells ros2 topic pub to publish the command in a steady stream at 1 Hz.

Finally, you can run echo on the pose topic:

```
ros2 topic echo /turtle1/pose
```

You can see that the `/turtlesim` node is also publishing to the pose topic.

3.3.6 Measuring the publication rate

The command `ros2 topic hz` estimates the rate at which messages are received.

```
ros2 topic hz /turtle1/pose
```

The result depends on the simulator and the execution environment.

You can also measure the rate of velocity commands:

```
ros2 topic hz /turtle1/cmd_vel
```

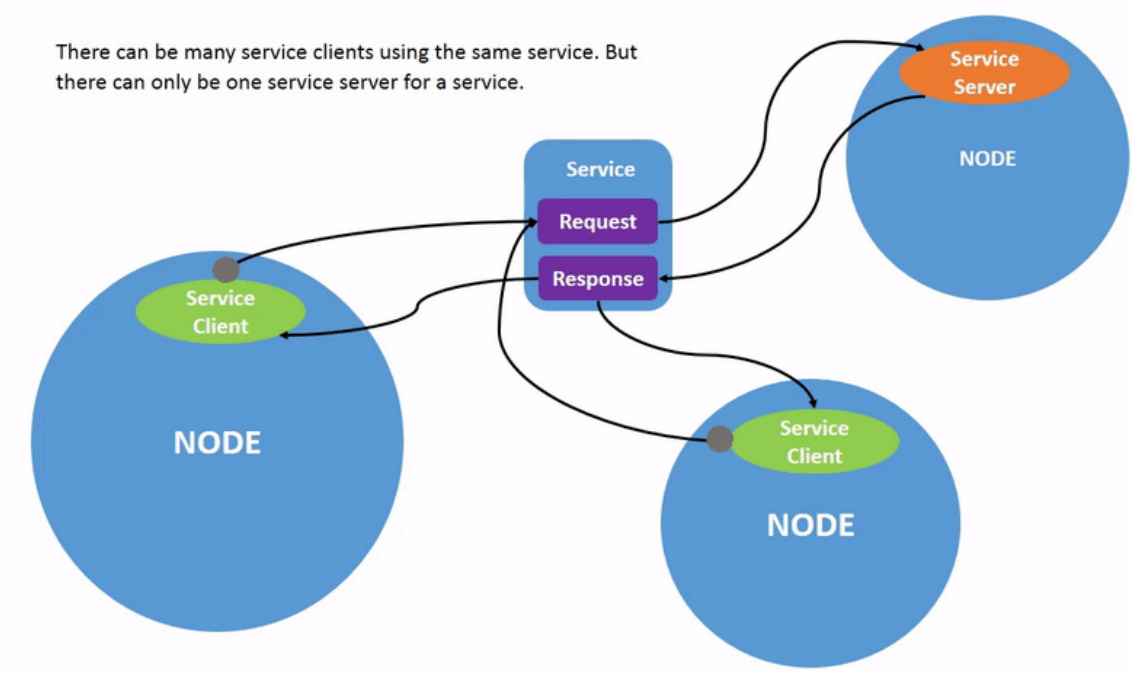
If messages are published at 1 Hz, the measured rate should be approximately one message per second.

3.4 Services

Services are another method of communication for nodes in the ROS graph.

A service is a request/response pattern where a client sends a request to a service server, which processes the request and returns a response.

Unlike topics, which are generally used to stream data, services are suitable for operations that require a response.



3.4.1 Listing services

Display the available services:

```
ros2 service list
```

The turtlesim simulator provides services such as:

```
/clear
/kill
/reset
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
```

3.4.2 Inspecting a service type

Services have types that describe how the request and response data of a service is structured.

Service types are defined similarly to topic types, except service types have two parts : one message for the request and another for the response.

To find out the type of a service, use the command:

```
ros2 service type <service_name>
```

To inspect the type of the `/spawn` service:

```
ros2 service type /spawn
```

The result should be:

```
turtlesim/srv/Spawn
```

To see the types of all the active services at the same time, you can append the `-show-types` option, abbreviated as `-t`, to the list command:

```
ros2 service list -t
```

Which will return:

```
/clear [std_srvs/srv/Empty]
/kill [turtlesim/srv/Kill]
/reset [std_srvs/srv/Empty]
/spawn [turtlesim/srv/Spawn]
...
/turtle1/set_pen [turtlesim/srv/SetPen]
/turtle1/teleport_absolute [turtlesim/srv/TeleportAbsolute]
/turtle1/teleport_relative [turtlesim/srv/TeleportRelative]
...
```

```
ros2 interface show turtlesim/srv/Spawn
```

The request contains the desired turtle position, orientation, and optional name. The response returns the name assigned to the new turtle.

3.4.3 Calling a service

You can call services from the command line, but first you need to know the structure of the input arguments.

```
ros2 interface show <type_name>
```

From the results of `ros2 service list -t`, we know `/spawn`'s type is `turtlesim/srv/Spawn`. To see the request and response arguments of the `/spawn` service, run the command:

```
ros2 interface show turtlesim/srv/Spawn
```

Which will return:

```
float32 x
float32 y
float32 theta
string name # Optional. A unique name will be created and returned if
this is empty
---
string name
```

The information above the `--` line tells us the arguments needed to call `/spawn`. `x`, `y` and `theta` determine the 2D pose of the spawned turtle, and `name` is clearly optional.

The information below the line isn't something you need to know in this case, but it can help you understand the data type of the response you get from the call.

Now that you know what a service type is, and how to find the structure of that type's arguments, you can call a service using:

```
ros2 service call <service_name> <service_type> <arguments>
```

The `<arguments>` part is optional. For example, the Empty typed services don't have any arguments:

```
ros2 service call /clear std_srvs/srv/Empty
```

This command will clear the turtlesim window of any lines your turtle has drawn.

Now let's spawn a new turtle by calling `/spawn` and setting arguments.

You can guess from its name that `/spawn` will create another turtle in the turtlesim window. Input `<arguments>` in a service call from the command-line need to be in YAML syntax.

Enter the command:

```
ros2 service call /spawn turtlesim/srv/Spawn \
"{x: 2.0, y: 2.0, theta: 0.2, name: 'turtle2'}"
```

A new turtle should appear in the simulator.

To remove it, call the `/kill` service:

```
ros2 service call /kill turtlesim/srv/Kill \
"{name: 'turtle2'}"
```

3.5 Parameters

A **parameter** is a named configuration value associated with a ROS 2 node.

Parameters can be used to configure a node's behavior without modifying its source code.

For example, turtlesim has parameters that control its background color.

3.5.1 Listing parameters

To see the parameters belonging to your nodes, open a new terminal and enter the command:

```
ros2 param list /turtlesim
```

You will see the node namespaces, `/teleop_turtle` and `/turtlesim`, followed by each node's parameters.

The output includes parameters similar to:

```
background_b
background_g
background_r
use_sim_time
```

3.5.2 Reading a parameter

To display the type and current value of a parameter, use the command:

```
ros2 param get <node_name> <parameter_name>
```

Let's find out the current value of `/turtlesim`'s parameter `background_g`:

```
ros2 param get /turtlesim background_g
```

Which will return the value:

```
Integer value is: 86
```

Now you know `background_g` holds an integer value. If you run the same command on `background_r` and `background_b`, you will get the values 69 and 255, respectively.

3.5.3 Changing a parameter

To change a parameter's value at runtime, use the command:

```
ros2 param set <node_name> <parameter_name> <value>
```

Let's change `/turtlesim`'s background color:

```
ros2 param set /turtlesim background_r 150
```

The background of your turtlesim window should change colors.

Parameters can also be set when a node starts:

```
ros2 run turtlesim turtlesim_node \
    --ros-args -p background_r:=255
```

Setting parameters with the set command will only change them in your current session, not permanently. However, you can save your settings and reload them the next time you start a node.

3.5.4 Dumping parameters

You can view all of a node's current parameter values by using the command:

```
ros2 param dump <node_name>
```

The command prints to the standard output (stdout) by default but you can also redirect the parameter values into a file to save them for later:

```
ros2 param dump /turtlesim > turtlesim.yaml
```

You will find a new file in the current working directory your shell is running in. If you open this file, you'll see the current configuration of `/turtlesim`'s parameters.

Dumping parameters comes in handy if you want to reload the node with the same parameters in the future.

3.5.5 Loading parameters

You can load parameters from a file to a currently running node using the command:

```
ros2 param load <node_name> <parameter_file>
```

To load the `turtlesim.yaml` file generated with `ros2 param dump` into `/turtlesim` node's parameters, enter the command:

```
ros2 param load /turtlesim turtlesim.yaml
```

Note : Read-only parameters can only be modified at startup and not afterwards, that is why there are some warnings for the "qos_overrides" parameters.

To start the same node using your saved parameter values, use:

```
ros2 run <package_name> <executable_name> --ros-args --params-file <file_name>
```

This is the same command you always use to start turtlesim, with the added flags `-ros-args` and `-params-file`, followed by the file you want to load.

Stop your running turtlesim node, and try reloading it with your saved parameters, using:

```
ros2 run turtlesim turtlesim_node --ros-args --params-file turtlesim.yaml
```

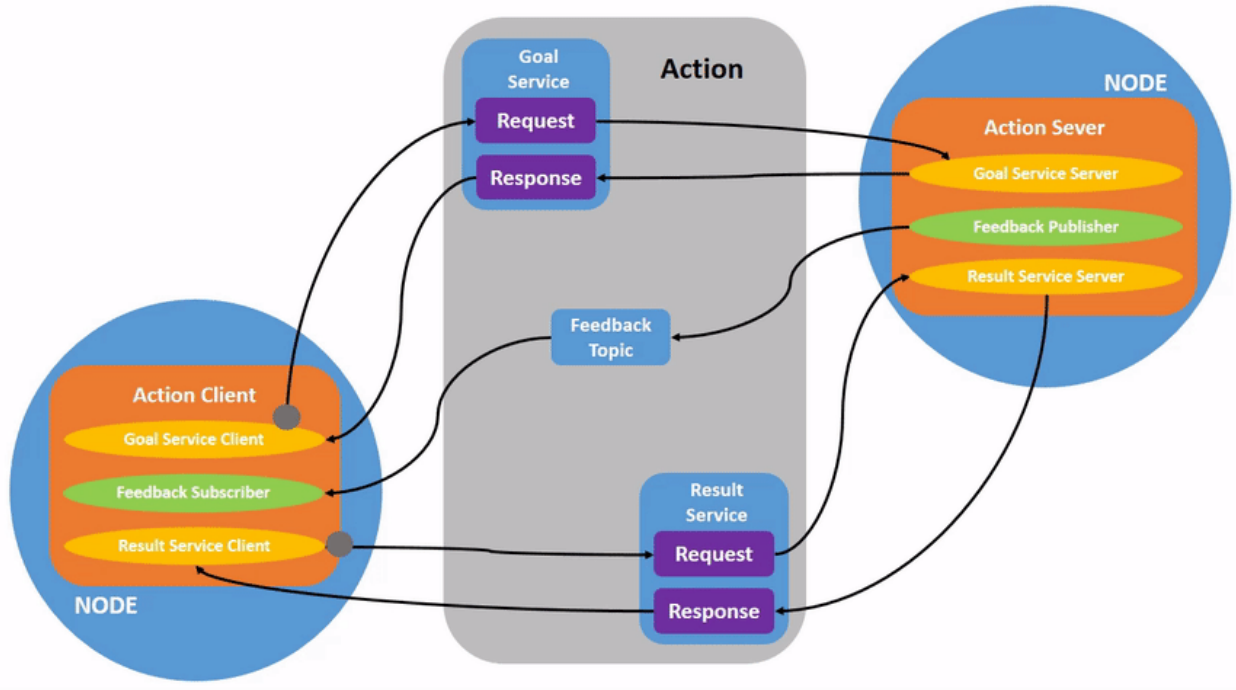
The turtlesim window should appear as usual, but with the purple background you set earlier.

3.6 Actions

Actions are one of the communication mechanism in ROS 2 and are intended for long running operations.

They are built on topics and services and they consist of three parts: a goal, feedback, and a result.

Actions are useful for tasks such as navigation, trajectory execution, and robot manipulation.



An action client sends a goal to an action server. The server can provide feedback while executing the goal and eventually return a result.

For more background on actions, see:

https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html

4. Creating and Building ROS 2 Packages

4.1 Creating a Workspace

A ROS 2 workspace is a directory containing ROS 2 packages. Before using ROS 2, it is necessary to source the ROS 2 installation environment.

```
source /opt/ros/jazzy/setup.bash
```

ROS 2 also supports the use of **overlays** and **underlays**. An underlay is an existing workspace that provides packages and dependencies, while an overlay is a secondary workspace where you can develop or add packages without modifying the original installation.

Packages in an overlay take precedence over packages with the same name in the underlay. The underlay must contain the dependencies required by the packages in the overlay.

Using overlays is recommended when working on a small number of packages, as it avoids rebuilding an entire workspace during each development iteration.

4.1.1 Create a New Directory

It is good practice to create a separate directory for each workspace. For this lab, we will use the name `ros2_ws`.

Open a terminal and execute the following commands:

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
```

The first command creates the workspace directory and its `src` subdirectory. The second command navigates into `src`, where ROS 2 packages will be created.

The workspace structure is initially as follows:

```
ros2_ws/
|-- src/
```

4.1.2 Clone a Sample Repository

A repository may contain several branches corresponding to different ROS 2 distributions. It is important to select the branch compatible with the installed distribution.

In this lab, we use ROS 2 Jazzy. From the `ros2_ws/src` directory, execute:

```
git clone https://github.com/ros/ros_tutorials.git -b jazzy
```

The repository contains the `turtlesim` package, which will be used in subsequent exercises. Other packages in the repository may contain a `COLCON_IGNORE` file, indicating that they should not be built by `colcon`.

4.1.3 Resolve Dependencies

Before building a workspace, it is important to check whether all required package dependencies are installed.

Navigate to the root of the workspace:

```
cd ~/ros2_ws
```

Then execute:

```
rosdep install -i --from-path src --rosdistro jazzy -y
```

If all dependencies are already installed, the terminal should display a message similar to the following:

```
#All required rosdeps installed successfully
```

Package dependencies are declared in the `package.xml` file. The `rosdep` command examines these declarations and installs any missing system dependencies.

4.1.4 Build the Workspace with Colcon

From the root of the workspace, build the packages using `colcon`:

```
cd ~/ros2_ws
colcon build
```

The terminal displays the build progress and a summary of the successfully built packages.

For example, the output may resemble:

```
Starting >>> turtlesim
Finished <<< turtlesim [5.49s]
Summary: 1 package finished
```

The exact output and build time may differ depending on the system.

After a successful build, list the contents of the workspace:

```
ls ~/ros2_ws
```

The workspace should contain the following directories:

```
build/
install/
log/
src/
```

Their purposes are as follows:

- `build/`: contains intermediate build files.
- `install/`: contains installed packages and setup scripts.
- `log/`: contains build logs.
- `src/`: contains the source code of the packages.

The `install/` directory contains the setup files needed to source the workspace as an overlay.

4.1.5 Source the Overlay

After building the workspace, the overlay must be sourced before its packages can be used.

It is recommended to open a new terminal before sourcing the overlay. This helps avoid potential environment conflicts between the terminal used for building and the terminal used for running ROS 2 applications.

In the new terminal, execute:

```
cd ~/ros2_ws
source install/setup.bash
```

You can now run the `turtlesim` node:

```
ros2 run turtlesim turtlesim_node
```

If the package exists in both the overlay and the underlay, ROS 2 will use the version found in the overlay.

4.1.6 Modify the Overlay

To observe the effect of an overlay, you can modify the source code of a package in the workspace.

For example, if the `turtlesim` source code is available in your workspace, navigate to its source directory:

```
cd ~/ros2_ws/src/ros_tutorials/turtlesim/src
```

Open the file `turtle_frame.cpp` using your preferred text editor.

Locate the following statement:

```
setWindowTitle("TurtleSim");
```

Change the window title to:

```
setWindowTitle("MyTurtleSim");
```

Save the file.

Return to the terminal used for building the workspace and rebuild the package:

```
cd ~/ros2_ws
colcon build --packages-select turtlesim
```

Open the terminal where the overlay is sourced and run the node again:

```
ros2 run turtlesim turtlesim_node
```

If the source modification was successfully compiled, the window title should display `MyTurtleSim`.

This example illustrates two important properties of overlays:

- Packages in an overlay can be modified and rebuilt independently of the underlay.
- The overlay takes precedence over the underlay when both contain the same package.

To verify that the underlay remains unchanged, open a new terminal, source the ROS 2 installation environment, and run:

```
source /opt/ros/jazzy/setup.bash
ros2 run turtlesim turtlesim_node
```

The original version of the package should run, provided that the installation and workspace are configured as described.

If you want to source the overlay automatically whenever a new terminal is opened, add the following command to your `.bashrc` file:

```
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
```

4.2 Creating a Package: Writing a Simple Publisher and Subscriber

A package is an organizational unit for ROS 2 code. Packages make it easier to organize, build, install, share, and reuse software.

ROS 2 officially supports packages built with both CMake and Python. The required files depend on the selected build system.

A typical CMake package contains:

- `CMakeLists.txt`: describes how to build the code within the package.
- `package.xml`: contains package meta information about the package and dependencies.
- `include/`: directory containing the public headers for the package.
- `src/`: contains the C++ source files.

A workspace can contain multiple packages, including packages that use different build systems (CMake, Python, etc.). However, packages should not be nested inside other packages.

For example, a workspace containing C++ and Python packages may have the following structure:

```
ros2_ws/
|-- src/
|   |-- cpp_package_1/
|   |   |-- CMakeLists.txt
|   |   |-- package.xml
|   |   |-- include/
|   |   `-- src/
|   |
|   |-- py_package_1/
|   |   |-- package.xml
|   |   |-- setup.py
|   |   |-- setup.cfg
|   |   |-- resource/
|   |   `-- py_package_1/
|   |
|   `-- cpp_package_2/
|       |-- CMakeLists.txt
|       `-- package.xml
|-- build/
|-- install/
`-- log/
```

4.2.1 Create a Package

In this section, you will create a C++ package containing two ROS 2 nodes: a publisher and a subscriber.

The publisher sends string messages over a topic, while the subscriber receives and displays those messages. This example illustrates how ROS 2 nodes communicate through the publish-subscribe mechanism.

The official ROS 2 examples are available at:

https://github.com/ros2/examples/tree/jazzy/rclcpp/topics

First, navigate to the source directory of your workspace:

```
cd ~/ros2_ws/src
```

The general syntax for creating a CMake-based ROS 2 package is:

```
ros2 pkg create --build-type ament_cmake \
    --license Apache-2.0 package_name
```

Create the package named `cpp_pubsub`:

```
ros2 pkg create --build-type ament_cmake \
    --license Apache-2.0 cpp_pubsub
```

The command creates the package directory and its initial files.

The resulting structure includes:

```
cpp_pubsub/
|-- CMakeLists.txt
|-- package.xml
‘-- include/
```

The exact directory structure may vary slightly depending on the ROS 2 tools and package creation options.

4.2.2 Write the Publisher Node

The publisher node will periodically send string messages over a topic.

Navigate to the package's source directory:

```
cd ~/ros2_ws/src/cpp_pubsub
mkdir -p src
cd src
```

Download the publisher example from the ROS 2 examples repository:

```
wget -O publisher_member_function.cpp \
https://raw.githubusercontent.com/ros2/examples/jazzy/rclcpp/topics/minimal_publisher/
    member_function.cpp
```

The file `publisher_member_function.cpp` contains the implementation of the publisher node.

Open the file using your preferred text editor and examine its contents.

For additional explanations, consult the official tutorials:

- C++ publisher and subscriber: https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Cpp-Publisher-And-Subscriber.html
- Python publisher and subscriber: https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html

4.2.3 Add Dependencies

Navigate to the package directory:

```
cd ~/ros2_ws/src/cpp_pubsub
```

Open `package.xml`.

Make sure that the package description, maintainer, and license information are correctly specified. For example:

```
<description>
  Examples of a minimal publisher and subscriber using rclcpp
</description>
<maintainer email="you@email.com">Your Name</maintainer>
<license>Apache-2.0</license>
```

Add the following dependencies inside the `<package>` element, alongside the existing dependency declarations:

```
<depend>rclcpp</depend>
<depend>std_msgs</depend>
```

The `rclcpp` dependency provides the C++ client library for ROS 2. The `std_msgs` dependency provides standard message types, including `std_msgs/msg/String`.

4.2.4 Configure CMakeLists.txt

Open the `CMakeLists.txt` file in the `cpp_pubsub` directory.

After the existing declaration

```
find_package(ament_cmake REQUIRED)
```

add the following dependency declarations:

```
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)
```

Next, register the publisher executable:

```
add_executable(talker src/publisher_member_function.cpp)

ament_target_dependencies(talker rclcpp std_msgs)
```

Install the executable so that it can be launched using `ros2 run`:

```
install(TARGETS
  talker
  DESTINATION lib/${PROJECT_NAME}
)
```

Make sure that these declarations are placed before the final `ament_package()` call in `CMakeLists.txt`.

You may remove unnecessary default comments and sections if they are not needed for this exercise.

4.2.5 Write the Subscriber Node

The subscriber node receives messages published on the same topic.

Navigate to the package's source directory:

```
cd ~/ros2_ws/src/cpp_pubsub/src
```

Download the subscriber example:

```
wget -O subscriber_member_function.cpp \
https://raw.githubusercontent.com/ros2/examples/jazzy/rclcpp/topics/minimal_subscriber/
    member_function.cpp
```

The source directory should now contain the two files:

```
publisher_member_function.cpp
subscriber_member_function.cpp
```

Open `subscriber_member_function.cpp` and examine the code.

The publisher and subscriber must use compatible message types and the same topic name to communicate successfully.

Unlike the publisher, which periodically sends messages, the subscriber processes messages when they arrive. The subscriber remains active while its ROS 2 node is spinning and waiting for incoming messages.

4.2.6 Configure the Subscriber Executable

The subscriber uses the same dependencies as the publisher. Therefore, no additional dependencies are required in `package.xml`.

Reopen `CMakeLists.txt` and add the subscriber executable after the publisher declarations:

```
add_executable(listener src/subscriber_member_function.cpp)

ament_target_dependencies(listener rclcpp std_msgs)

install(TARGETS
  talker
  listener
  DESTINATION lib/${PROJECT_NAME}
)
```

Ensure that the file contains only one appropriate `install(TARGETS ...)` section for these two executables. Save the file before proceeding.

4.2.7 Build the Package

Navigate to the root of the workspace:

```
cd ~/ros2_ws
```

Before building, check whether any dependencies are missing:

```
rosdep install -i --from-path src --rosdistro jazzy -y
```

Build the workspace:

```
colcon build
```

To build only the `cpp_pubsub` package, use:

```
colcon build --packages-select cpp_pubsub
```

The latter command is useful when modifying a single package in a larger workspace.

4.2.8 Source the Setup File

After a successful build, source the workspace so that ROS 2 can locate the newly created package and its executables.

From the workspace root, execute:

```
source install/setup.bash
```

This command makes the package available in the current terminal.

4.2.9 Run the Publisher and Subscriber

Step 1: Start the publisher. In the first terminal, source the ROS 2 installation and workspace, then launch the publisher:

```
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run cpp_pubsub talker
```

The publisher should start sending messages periodically. The terminal output will resemble the following:

```
[INFO] [minimal_publisher]: Publishing: "Hello World: 0"
[INFO] [minimal_publisher]: Publishing: "Hello World: 1"
[INFO] [minimal_publisher]: Publishing: "Hello World: 2"
[INFO] [minimal_publisher]: Publishing: "Hello World: 3"
[INFO] [minimal_publisher]: Publishing: "Hello World: 4"
```

The message counter increases as the publisher continues running.

Step 2: Start the subscriber. Open a second terminal. Source the ROS 2 installation and workspace:

```
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
```

Then start the subscriber:

```
ros2 run cpp_pubsub listener
```

The subscriber should display messages received from the publisher. For example:

```
[INFO] [minimal_subscriber]: I heard: "Hello World: 10"
[INFO] [minimal_subscriber]: I heard: "Hello World: 11"
[INFO] [minimal_subscriber]: I heard: "Hello World: 12"
[INFO] [minimal_subscriber]: I heard: "Hello World: 13"
```

The exact counter values depend on when the subscriber starts.

Step 3: Stop the nodes. To stop each node, press `Ctrl+C` in its terminal.

You can explore additional implementations in the official ROS 2 examples repository:

https://github.com/ros2/examples/tree/jazzy/rclcpp/topics

5. Launching Nodes

Launch files allow you to start and configure multiple executables containing ROS 2 nodes simultaneously. Using the `ros2 launch` command, you can start an entire system, including its nodes and configurations, from a single launch file.

5.1 Launching Multiple Turtlesim Nodes

Open a new terminal and execute:

```
ros2 launch turtlesim multisim.launch.py
```

If the launch file is available in your installation, this command starts two `turtlesim` nodes.

The launch file is written in Python and has the following structure:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            namespace='turtlesim1',
            package='turtlesim',
            executable='turtlesim_node',
            output='screen'
        ),
        Node(
            namespace='turtlesim2',
            package='turtlesim',
            executable='turtlesim_node',
            output='screen'
        ),
    ])
```

Open a second terminal and publish a velocity command to the first turtle:

```
ros2 topic pub /turtlesim1/turtle1/cmd_vel \
geometry_msgs/msg/Twist \
"{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

Open a third terminal and publish a velocity command to the second turtle:

```
ros2 topic pub /turtlesim2/turtle1/cmd_vel \
geometry_msgs/msg/Twist \
"{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: -1.8}}"
```

Both commands continuously publish velocity messages by default. The first turtle moves forward while turning in one direction, whereas the second moves forward while turning in the opposite direction.

Note: ROS 2 launch files can be written in Python, XML, or YAML. For more information about the different formats, consult the official documentation:

https://docs.ros.org/en/jazzy/How-To-Guides/Launch-file-different-formats.html

6. Recording and Playing Back Data

The `ros2 bag` command-line tool allows you to record messages published on ROS 2 topics and save them for later analysis or playback.

Recorded data is stored in a bag directory containing metadata and the recorded data files. These recordings can be replayed to reproduce experiments, analyze robot behavior, and share results with others.

First, open a terminal and start the `turtlesim` node:

```
ros2 run turtlesim turtlesim_node
```

Open a second terminal and start the keyboard teleoperation node:

```
ros2 run turtlesim turtle_teleop_key
```

6.1 Recording Data with ros2 bag

Create a directory to store the recordings. Open another terminal and execute:

```
mkdir -p ~/bag_files
cd ~/bag_files
```

The `ros2 bag` tool records messages published on topics. In this example, the teleoperation node publishes velocity commands on the `/turtle1/cmd_vel` topic.

To observe these messages, open another terminal and run:

```
ros2 topic echo /turtle1/cmd_vel
```

Use the arrow keys to move the turtle. You should see velocity messages displayed in the terminal running `ros2 topic echo`.

Press `Ctrl+C` to stop the command.

6.1.1 Recording a Single Topic

The general syntax for recording messages published on a topic is:

```
ros2 bag record <topic_name>
```

Open a new terminal and navigate to the directory created earlier:

```
cd ~/bag_files
```

Start recording the velocity commands:

```
ros2 bag record /turtle1/cmd_vel
```

The terminal displays messages indicating that recording has started. The exact output depends on your ROS 2 version and storage configuration.

For example, you may see messages indicating that the tool has opened a storage file, subscribed to the selected topic, and started recording messages.

Return to the teleoperation terminal and move the turtle around. Try to create a recognizable trajectory so that you can compare it with the trajectory obtained during playback.

When you have finished, press `Ctrl+C` in the recording terminal.

The recording is saved in a directory whose name is generated automatically. It typically contains a `metadata.yaml` file and a data file, such as a `.db3` file when SQLite storage is used.

6.1.2 Recording Multiple Topics

You can record several topics simultaneously and specify a custom name for the recording.

For example, to record the velocity commands and the turtle pose, execute:

```
cd ~/bag_files
ros2 bag record -o subset \
/turtle1/cmd_vel /turtle1/pose
```

The `-o` option specifies the output name. In this example, the recording is named `subset`.

Both topics are recorded in the same bag. Move the turtle around and press `Ctrl+C` when you have collected enough data.

To record all available topics, you can use the `-a` option:

```
ros2 bag record -a
```

Note: Recording all topics may generate large files, depending on the number of topics and their publication rates.

6.2 Inspecting a Recording with ros2 bag info

The `ros2 bag info` command displays information about a recording, including its duration, storage format, message count, and topic information.

The general syntax is:

```
ros2 bag info <bag_file_name>
```

To inspect the recording created in the previous example, run:

```
cd ~/bag_files
ros2 bag info subset
```

The output provides information similar to the following:

```
Files:             subset.db3
Bag size:          ...
Storage id:        sqlite3
Duration:          ...
Start:             ...
End:               ...
Messages:          ...

Topic information:
  Topic: /turtle1/cmd_vel
  Type: geometry_msgs/msg/Twist
  Count: ...

  Topic: /turtle1/pose
```

```
  Type: turtlesim/msg/Pose
  Count: ...
```

6.3 Playing Back Recorded Data

Before replaying the recording, stop the keyboard teleoperation node by pressing `Ctrl+C` in its terminal. Keep the `turtlesim` node running so that you can observe the playback.

Navigate to the directory containing the recording:

```
cd ~/bag_files
```

Replay the recorded data:

```
ros2 bag play subset
```

The tool reads the messages stored in the bag and republishes them on their original topics.

If the recording contains messages on `/turtle1/cmd_vel`, the turtle will reproduce the recorded movements, provided that the corresponding `turtlesim` node is running.

The trajectory may not be reproduced with perfect accuracy because the simulation can be sensitive to timing differences.

You can stop playback at any time by pressing `Ctrl+C`.

7. Using rqt, rqt_graph, and rqt_console

`rqt` is a graphical user interface (GUI) that provides plugins for inspecting and interacting with ROS 2 applications.

It includes tools for visualizing node connections, monitoring messages, and viewing log messages.

7.1 Installing rqt

Install the required packages:

```
sudo apt update
sudo apt install '~nros-jazzy-rqt*'
```

Start the graphical interface:

```
rqt
```

The available plugins can be accessed through the menu.

7.2 Use rqt

When running `rqt` for the first time, its window may appear blank. To open the Service Caller plugin, select the following menu option:

`Plugins` → `Services` → `Service Caller`

Note: It may take some time for `rqt` to discover all available plugins. If you open the `Plugins` menu but cannot find the `Services` category or other expected options, close `rqt` and run the following command in a terminal:

```
rqt --force-discover
```

Once the Service Caller plugin is open, click the refresh button located to the left of the Service dropdown list. This ensures that the available services provided by the running `turtlesim` node are refreshed.

Click the Service dropdown list to display the available services, then select:

```
/spawn
```

The `/spawn` service allows you to create a new turtle in the simulation by specifying its initial position, orientation, and name.

Ensure that the `turtlesim` node is running before proceeding to the next step.

7.3 Visualizing the ROS Graph

Start turtlesim and the keyboard controller.

Then execute:

```
rqt_graph
```

Alternatively, open `rqt` and select:

`Plugins` → `Introspection` → `Node Graph`

The graph displays the nodes and their topic connections.

For example:

- `/teleop_turtle` publishes velocity commands to `/turtle1/cmd_vel`.
- `/turtlesim` subscribes to `/turtle1/cmd_vel`.
- `/turtlesim` publishes the turtle's pose on `/turtle1/pose`.

Use the graph to understand how data flows between the nodes.

7.4 Monitoring Logs with rqt_console

The `rqt_console` plugin displays log messages generated by ROS 2 nodes.

Launch it with:

```
ros2 run rqt_console rqt_console
```

You can also access it through the `rqt` plugin menu.

Inspect the messages produced by the running nodes and observe how log messages help with debugging.

References

- ROS 2 Jazzy Documentation: https://docs.ros.org/en/jazzy/
- ROS 2 Tutorials: https://docs.ros.org/en/jazzy/Tutorials.html
- Using turtlesim, ros2, and rqt: https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/Introducing-Turtlesim.html
- Understanding ROS 2 Topics: https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html